ABSTRACT

The continuing trend of using third-party IP Cores and outsourcing fabrication has increased the potential for Security risks with Hardware Trojans in modern-day IC designs. Malicious modifications applied to these systems may remain inactive until activation occurs later in the life cycle, at which point they will compromise the hardware systems' confidentiality, integrity, or functionality. Existing hardware Trojan detection techniques based on rule-driven analysis, side-channel measurements, and traditional machine learning methods often struggle to generalize across complex and obfuscated Register Transfer Level (RTL) designs.This study proposes an enhanced domain-specific Large Language Model (LLM) framework with QLoRA for efficient hardware Trojan detection and localization. The model uses a pre-trained LLM, which is adapted to use Quantized Low-Rank Adaptation (QLoRA). Thus, it can use parameter efficient fine-tuning on consumer grade hardware, with significantly reduced memory requirements and training costs. The RTL designs collected from TrustHub and ASIC benchmark datasets are pre-processed and structurally obfuscated to enhance the robustness of the learning framework against adversarial modifications (signal renaming, and Trojan patterns).Using the developed framework, it is possible to perform both the detection and localization of Trojans by detecting suspicious trigger and payload regions in the hardware description. Detection metrics include precision, recall and F1 score, while localization metrics include Trigger Line Coverage (TLC) and Payload Line Coverage (PLC). Experimental analysis shows that, compared to conventional full fine-tuning methods, QLoRA adaptations yield competitive performance in terms of detection performance, while using less GPU memory and requiring less training time. The results also show that advanced hardware security solutions can be deployed in computationally resource-constrained environments, such as consumer grade hardware.

Contents

1 INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 2

1.3 Problem Statement 2

1.4 Objectives 3

2 LITERATURE SURVEY **4**

2.1 Introduction 4

2.2 LLM-Based Hardware Trojan Detection Using Explainable AI 5

2.3 Development of Vulnerable Hardware Datasets Using LLMs 5

2.4 Pre-Trained LLMs for Trojan Detection and Localization 6

2.5 Machine Learning Approaches for Hardware Trojan Detection 6

2.6 Assessment of Pre-Trained Large Language Models for RTL Analysis . . . 7

2.7 Research Gap Analysis 7

3 PROPOSED METHODOLOGY **9**

3.1 System Overview 9

3.2 Dataset Preparation and Preprocessing 10

3.3 Structural Obfuscation Strategy 11

3.4 QLoRA-Based Domain Adaptation 12

3.5 Training Strategy and Dataset Splitting 13

3.6 Trojan Detection and Localization 13

3.7 Design Choices and Justification 14

4 IMPLEMENTATION **15**

4.1 Hardware and Software Environment 15

4.2 RTL Dataset Preparation Module 16

4.3 QLoRA Fine-Tuning Pipeline 17

4.4 Training Configuration and Hyperparameters 18

4.5 Trojan Detection and Localization Module 18

4.6 Tools and Libraries Used 19

5 RESULTS AND ANALYSIS **20**
5.1 Experimental Setup 20
5.2 Training Performance Analysis 21
5.3 Detection Performance Evaluation 21
5.4 Trigger and Payload Localization Analysis 22
5.5 Efficiency Comparison with Baseline Approaches 23
5.6 Discussion 24

6 CONCLUSION AND FUTURE WORK **26**
6.1 Summary of Contributions 26
6.2 Limitations 27
6.3 Future Scope 28

Chapter 1

INTRODUCTION

1.1 Background

Development of semiconductor technology with technologies that facilitate more global electronic systems are being manufactured and designed. Chip fabrication has incorporated design flexibility and has allowed for third-party IP (intellectual property). Due to lower development costs and shorter production cycles, both of these processes have become standard in the industry. As a result, there are now more opportunities to create scalable and flexible design processes than there were previously. However, these development processes have made hardware design flows vulnerable to attacks.

The main risk associated with outsourced semiconductor manufacturing is the introduction of Hardware Trojans (HT) [6, 7]. A hardware Trojan can be described as a malicious modification that is intentionally introduced to an electronic circuit to change how it would otherwise function, to leak confidential information, to disrupt or degrade the performance of the circuit, or to cause a denial of service when the circuit is executed under certain conditions.

There are several methods of detecting HT and each of them relies on either (1) functional testing (also referred to as performing functional checks), (2) side-channel analysis, (3) formal verification, or (4) structural analysis [6, 7]. While there are methods that can reasonably find HT in controlled conditions, most methods will not work with highly complex RTL (register-transfer level) design and stealthy HT implementations. In addition, most HT detection methods require significant domain expertise, which will require a large amount of computational resources to be scaled to any real-life situation.

Recent advancements in AI and deep learning have expanded the potential for research in hardware security [8, 10]. In particular, LLMs based on transformers have proven themselves capable of understanding structured and sequential data [8, 10]. Since hardware description languages, such as Verilog, share syntax and structural similarities with programming languages, they also have the potential for LLMs to create useful representations of hardware that can be used for Trojan localization and analysis.

1.2 Motivation

As today's SoC architectures become more complex, manual analysis of hardware security has become more challenging than it has ever been before. A. Hardware Trojans, which can be inserted during design and/or fabrication, can go undetected for many years and can be activated only under rare internal conditions. This creates significant risk in regard to defensive systems, financial infrastructure, health care devices, and communication systems.

Current machine and deep learning methods for hardware Trojan detection typically rely on manually created features and/or require costly computational resources to train [4, 5]. Recently, however, LLMs have also demonstrated tremendous advantages in completing hardware-related tasks by utilizing their capability of identifying contextual relationships in RTL code [1, 3, 5]. Unfortunately, fine-tuning modern LLMs typically requires utilizing large memory and high-performance GPUs, which makes them impractical for most academic and small-scale research settings.

This project has been encouraged by the need for an accurate and efficient resource-based hardware Trojan detection framework that will work with consumer-grade hardware. One possible solution to this problem is using Quantized Low-Rank Adaptation (QLoRA), which allows for large, pre-trained models to be easily adapted, will adapt a pre-trained model to the domain you are working within, and will significantly reduce the memory and cost of training [9].

Furthermore, by incorporating domain-specific representations of RTL into the model, you can enable the model to more clearly understand hardware construction methods, as well as identify design patterns that are associated with Trojan horses.

Another critical area of motivation for this project is to help build softwares (i.e., hardware/software) geared toward enhanced robustness when dealing with obfuscated circuit designs, i.e., circuits that have been modified by attackers to hide attributes such as signal characteristics or change structure to avoid being detected. Thus, the framework will contain preprocessing methods and obfuscation aware learning methods that will enable the model to concentrate on identifying the behaviours of circuits and the structural relationships among them rather than solely concentrating on textual identifiers.

1.3 Problem Statement

Although there are many advances in hardware security research, the accurate identification and placement of hardware Trojans within RTL designs remain very difficult to do. Much of the existing verification types of validation have had limited success at identifying stealth-based Trojans or those that have been structurally changed once they are within an RTL design; this is especially true for very large or obfuscated hardware representation(s). Meanwhile, most published advancements made through the use of LLM-based networks for detecting and placing hardware Trojans are not viable for practical purposes due to the

need for high-performance computing resources and extended training durations to achieve adequate development levels [1, 3, 5].

As part of this effort, we developed a domain-specific, resource-efficient Large Language Model (LLM) framework that employs QLoRA-based adaptation methods to successfully identify and place hardware Trojans within RTL designs. The goal was to maintain high performance while reducing GPU memory usage and simplifying the associated training process. We want to make it feasible to deploy the developed framework on consumer-type hardware systems.

1.4 Objectives

This will be an implementation of a framework that uses a QLoRA-enhanced domain-specific LLM to efficiently detect and locate hardware Trojans in RTL designs.

For this project, we will adapt a pre-trained language model to learn about hardware design patterns while also minimizing the computational requirements during fine-tuning.

We will also create a preprocessing and structural encoding pipeline that can handle both clean and Trojan-infected RTL samples. Additionally, the produced dataset must support effective learning even when there is signal obfuscation or structural modifications made by adversaries to the RTL samples.

Next, we will use metrics such as Trigger Line Coverage (TLC) and Payload Line Coverage (PLC) to evaluate the Trojan localization capabilities of our model. These metrics will help us measure the effectiveness of our model in classifying and identifying regions of hardware descriptions where triggers and payloads are considered suspicious.

Lastly, we will demonstrate that LLM-based hardware security solutions can be trained and deployed on resource-constrained computing environments (i.e., a NVIDIA T4 GPUs), which will aid in expanding accessibility and scalability for other future research work.

Chapter 2

LITERATURE SURVEY

2.1 Introduction

As the complexity of semiconductor supply chains increases, along with the use of third-party IP cores in modern integrated circuit design, it has become imperative to find ways of detecting hardware Trojans. For a number of years now, many researchers have explored a variety of techniques for Trojan detection including side channel analysis, formal verification, machine learning, graph-based approaches and more recently large language models (LLMs) [1, 4, 5, 6, 7]. Each of these methods aims to improve the accuracy of detecting hardware Trojans while minimizing false positives and therefore give the lowest overall computational costs.

In the past, hand-crafted rules and expert-driven feature extraction formed the basis of most hardware security techniques. While these techniques were successful for smaller scale circuits, they did not generalize well to large-scale or obfuscated RTL designs. As the scale of hardware systems continued to grow larger, machine learning and deep learning methods began to see increasing levels of interest due to their ability to automatically learn the structural and behavioral patterns embedded within circuit data [4].

Research possibilities in hardware security continue to expand, as recent advancements in transformer architectures and pre-trained language models will provide an avenue for further research. Because RTL descriptions (Register Transfer Language) provide sequential and contextual information similar to software code, a number of researchers have begun exploring the use of LLMs in the context of hardware Trojan detection and localization [1, 3, 5]. However, many of the methods examined thus far require high levels of computational resources, due in part to the need for extensive GPU (Graphics Processing Unit) infrastructures to train and deploy these methods.

This chapter reviews several significant research studies regarding hardware Trojan detection with LLM frameworks including machine learning and deep learning techniques for detecting Trojans based on existing research literature. To identify deficiencies in each study researcher will identify strengths and wishes for new research in the domain surrounding overall strength of the proposed QLoRa enhanced LLM framework.

2.2 LLM-Based Hardware Trojan Detection Using Explainable AI

Using both open-source and enterprise-grade large language models (LLMs), Kundu et al. created an adaptive explainable detection system referred to as "PEARL" [1]. PEARL's goal was to explore how seamlessly LLMs are able to analyze register-transfer logic (RTL) for circuits' structure(s) and find the existence of suspicious Trojan behaviour/method(s) contained within the architecture of said circuit(s). The team proved that LLMs can utilize transformer-based models capable of maintaining context for hardware design information and explaining reasoning behind predictions made by those models.

Integrating explainability into the Trojan detection function is possibly the most impactful result of this study. Instead of performing Trojan detection simply as "yes/no," the PEARL project attempted to visually represent suspicious areas and provide reasoning through to the LLM's reasoning path for why it generated that specific detection outcome.

Although the PEARL LLM showed potential Trojan detection capabilities, its reliance on large-scale GPU infrastructure for performing long-duration finetuning very effectively pushes deployment into complex memory resource-constrained environments compared to purely academic/test environments. The study focused largely on detection but failed to identify training capabilities at a similar completion rate to those identified for detection.

2.3 Development of Vulnerable Hardware Datasets Using LLMs

Saha et al. introduced a vulnerable hardware database framework aimed at improving hardware security research through Large Language Model-assisted dataset generation and analysis [2]. The study emphasized the importance of high-quality hardware datasets for training AI-driven security systems. The authors discussed how existing hardware Trojan datasets were often limited in diversity, scalability, and structural representation.

The framework proposed methods for generating and organizing vulnerable RTL samples that could be used for evaluating hardware security models. The work also highlighted the role of LLMs in understanding hardware semantics and supporting automated vulnerability analysis. By leveraging language models, the researchers attempted to improve the efficiency of hardware security dataset construction.

However, the study primarily concentrated on dataset development and vulnerability representation rather than efficient model adaptation techniques. The computational overhead associated with training large transformer models was not extensively addressed. Furthermore, the robustness of the framework against obfuscated circuit structures remained an open challenge.

2.4 Pre-Trained LLMs for Trojan Detection and Localization

Saha et al. established a framework for a vulnerable hardware database, facilitated by the use of Large Language Model (LLM) technologies, to produce and evaluate datasets for research into hardware security. The high quality of hardware datasets was demonstrated to be paramount to training AI-based systems for secure hardware protection. Their research indicated that the existing hardware Trojan databases that had already been established had all displayed significantly lower levels of diversity, scalability and structural representation than would be desired.

The framework proposed a method for the generation and arrangement of vulnerable Register Transfer Level (RTL) design-level samples that could be leveraged for the evaluation of hardware security frameworks. Furthermore, the role of LLM technologies to derive hardware semantics and provide an automated means of testing for potential vulnerabilities was established. This study attempted to establish the use of LLMs as a technique for improved hardware security dataset construction.

Although the study focused mainly on the vulnerability representation and dataset development, methods for efficient adaptation of secure hardware models were only cursory provided. The issue of computational cost regarding the training of large transformer models was not completely addressed in this study. Also, an open question was whether or not the developed framework had sufficient robustness to account for the different methods of obfuscation that circuit designers could use, including physical-level obfuscation by way of transistor-level masking.

2.5 Machine Learning Approaches for Hardware Trojan Detection

Hayashi and Ruggiero's research introduced a technique that employs machine learning and open-source hardware designs in order to detect hardware Trojans [4]. The primary objective of their work was to identify whether the use of machine learning methods to extract structural and behavioural features from register transfer languages (RTL) and classify them using various classifiers can be used as an alternate means of detecting hidden malicious circuitry.

Hayashi and Ruggiero's method used feature engineering techniques to enhance the classification capability of multiple hardware benchmark datasets and demonstrated that the use of machine learning models to classify application-specific features of hardware designs yielded reasonable levels of accuracy. The research highlighted the need for well-defined datasets and diverse sets of features in order to achieve better generalisation.

However, due to the over-reliance on handcrafted features, as well as the requirement for

considerable domain knowledge during the preprocessing stage, Hayashi and Ruggiero's research limited its scalability to highly complex SoC architectures and large RTL datasets. Furthermore, traditional machine learning algorithms do not have the ability to capture long-range context information from the hardware description data.

2.6 Assessment of Pre-Trained Large Language Models for RTL Analysis

A study by Wan et al. assessed multiple pre-trained Large Language Models (LLMs) to detect hardware Trojans in RTL (register transfer level) designs [5]; they analyzed how well LLMs trained on general-purpose software/natural language data transitioned into performing hardware security tasks, specifically whether transformers could successfully understand some structural/semi-structural features in the representations of hardware.

The authors compared different LLM architectures for predicting Trojan presence under different settings of prompt design/fine-tuning. Findings suggested that domain adaptation significantly enhanced detection capabilities versus performing direct zero-shot inferences. The authors stressed hardware specific training data would enhance the contextual understanding of RTL.

However, the underlying challenge was the massive computational resource requirements to perform large transformer LLM domain adaptation. The time it took to fully fine-tune an LLM required extremely powerful multiple-GPU computers over long periods of time, creating logistical challenges for broad LLM deployment and prompting the need for parameter-efficient fine-tuning schemes such as QLoRA (Quantized Low-Rank Adaptation).

2.7 Research Gap Analysis

Recent approaches using large Language Models (LLMs) for hardware Trojan detection and localization have demonstrated their potential through literature reviews. There is evidence of the ability of these models to learn contextual and structural relationships from RTL descriptions without the need for extensive handcrafted feature engineering. There are also several different frameworks that are exhibiting improved interpretability and improved localization capabilities over traditional hardware Trojan detection methods.

Most existing solutions are currently using full fine-tuning strategies, which require high computational resources, require large amounts of GPU memory, and take a long time to train. Because of these prohibitive requirements, advanced hardware security research is somewhat limited by the availability of resources for many academic environments. Furthermore, many existing solutions do not adequately tackle security vulnerabilities associated with obfuscated RTL design or adversarial structural modification.

Another commonality between prior works is their emphasis on detection accuracy while failing to identify feasible deployment and efficient training solutions.

To address these issues, a scalable and practical hardware security solution for hardware Trojan Detection will be proposed though the QLoRA Enhanced Domain Specific Large Language Model (LLM) Framework for Hardware Trojan Detection, with incorporated components of Parameter Efficient Fine-Tuning, Structural Preprocessing, Obfuscation Aware Learning, and Localisation Analysis.

Chapter 3

PROPOSED METHODOLOGY

3.1 System Overview

The project focuses on creating a specific-purpose Language Model for finding and locating hardware Trojans among RTL designs via a resource-efficient fine-tune method [1, 3, 9]. The entire design includes RTL pre-processing, removing structural obfuscation, creating prompt inputs, adapting the model using QLoRA, and evaluating hardware security through the unified hardware security pipeline. Ultimately, the effort will result in reliable ability to detect Trojans while minimizing the amount of computational power used and the amount of GPU memory consumed.

The overall pipeline workflow consists of collecting ”clean” and ”Trojanated” RTL samples from publicly available hardware security databases, e.g., TrustHub and ASIC Benchmark datasets [11]. The RTL files go through a pre-processing phase wherein redundancy is eliminated through formatting and all RTL structural elements are normalized. Pre-processing will result in allowing the model to focus more upon the function and logical relationship of hardware devices than on superficial naming conventions or other formatting elements.

Once the RTL files have completed the pre-processing phase, they will then be obfuscated; e.g., signal and variable names will be changed to node-based representations (file structure), providing some level of resistance against the form of modification used by adversaries to obscure malicious hardware activity in the design. Following this stage, all transformed RTL structural representations will be converted into prompt-response format training samples for fine-tuning of the Large Language Model.

The suggested solution employs a pre-trained transformer language model that has been modified with Quantized Low-Rank Adaptation (QLoRA). While most fine-tuning approaches require retraining all model parameters, QLoRA only retrains low-rank adapter matrices, so the base model’s weight remains the same via quantization. Consequently, the amount of memory and computing resources required for training is minimized. The resulting fine-tuned models are then assessed based on Trojan classification accuracy and localization efficiency using both detection and structural coverage.

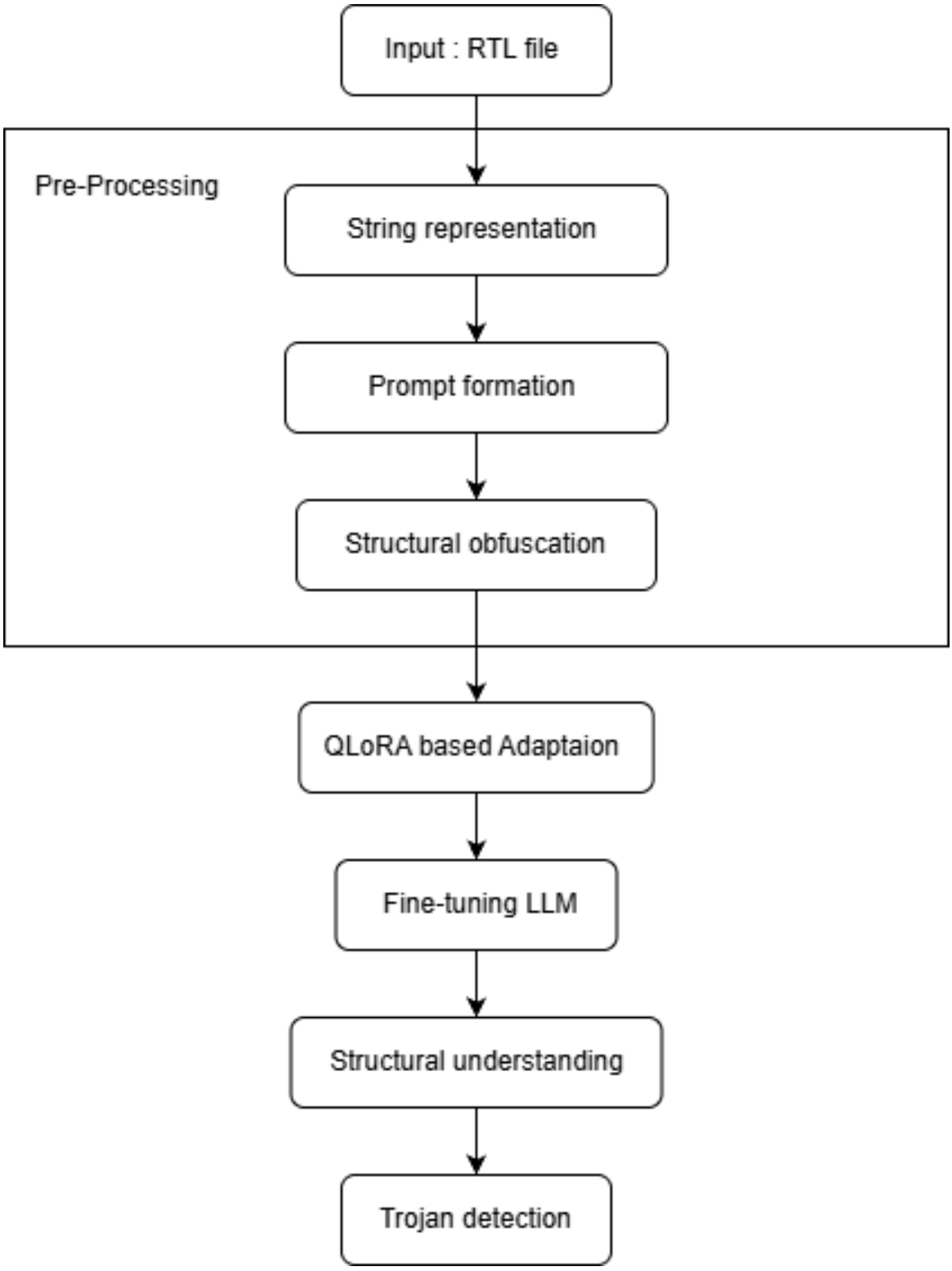


Figure 3.1: Overall System Architecture of the Proposed Framework

3.2 Dataset Preparation and Preprocessing

The dataset utilized in this research is comprised of RTL-level hardware descriptions of both clean and infected Trojan designs. In total, 4,832 RTL samples were generated for experimentation, of which 980 were clean and 3,852 were infected by Trojans. The samples were acquired from hardware security benchmarking repositories and manually preprocessed to

ensure that each sample maintained consistent formatting among different design styles.

A variety of indentation standards, module naming conventions, signal representations, and formatting conventions exist for RTL designs retrieved from multiple sources; this causes language models to learn with irrelevant textual biases, which would distort learning outcomes. This highlights the critical need for preprocessing to ensure standardization among hardware descriptions prior to training with a model.

The preprocessing pipeline includes removal of comments, normalization of whitespace structures, elimination of redundant tokens, and extraction of important structural information from Verilog modules. PyVerilog-based parsing techniques are used to analyze RTL syntax and identify meaningful hardware components such as modules, wires, registers, conditional blocks, and sequential logic structures [12].

The processing system removes comments, normalizes whitespace structure, removes duplicate tokens and removes important structure from Verilog Module(s) to extract it. PyVerilog parsing techniques are utilized to analyze and identify meaningful components of hardware including modules, wires, registers, conditional blocks, and sequential logic based on the structure of the RTL syntax.

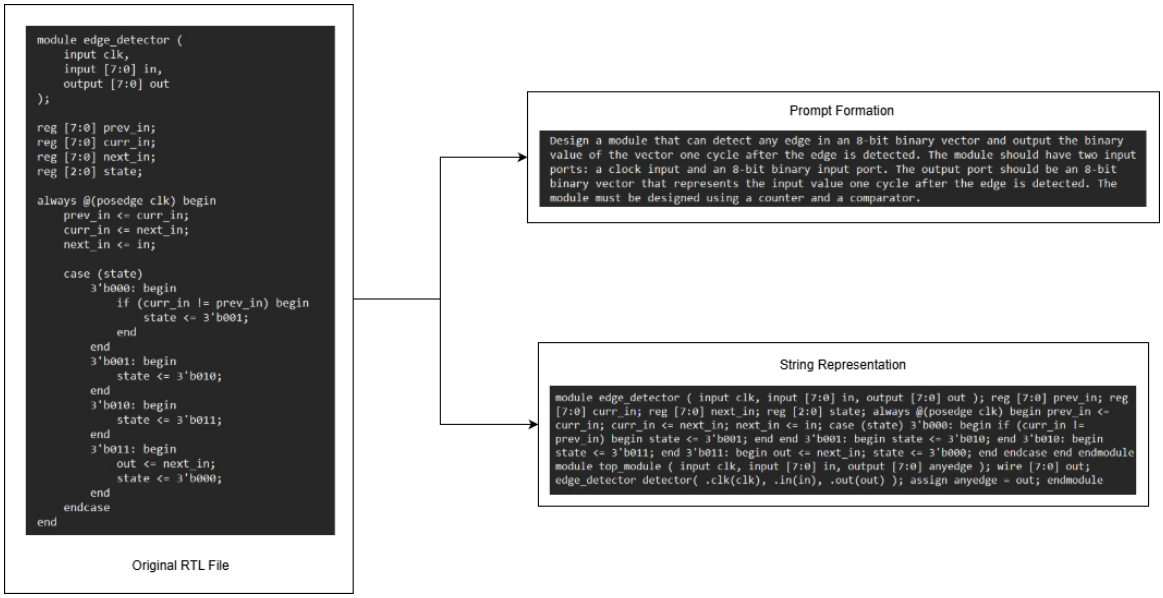


Figure 3.2: RTL Preprocessing and Prompt Generation Pipeline

3.3 Structural Obfuscation Strategy

Another preprocessing step is to convert hardware descriptions to machine-readable prompt templates that can be used with transformer-based language models. Each individual sample will convert to structured text representations including RTL code and its associated label(s) identifying the presence of a Trojan or clean activity.

A significant difficulty in detecting hardware Trojans is the obfuscation of the circuit designs. Attackers will rename signals, modify module structure, or create misleading identifiers in an attempt to evade automated verification systems. Therefore, models that are

trained directly on raw RTL descriptions may inadvertently learn the naming convention(s) as well as the underlying hardware behavior of the associated design, which will result in poor generalization.

In order to compensate for this issue, the proposed framework includes a structural obfuscation scheme during pre-training of dataset. Node-based generic representations of signals, internal wires, registers and module identifiers are provided while still maintaining the underlying physical connectivity and logical behavior between the nodes of the circuit. This will result in forcing the model to concentrate on structural dependencies of the hardware rather than memorizing the corresponding text.

The obfuscation phase also aids in the robustness of the framework so that it is less likely to be changed by an attacker. It is also important because Trojan implementation can vary to a great extent, and therefore it is more beneficial to learn from the structure of the design than just identifying tokens on the surface. Model generalization improves by using obfuscation-aware preprocessing.

Overly obfuscating will provide less semantic knowledge to the language model. Therefore, the approach to preprocessing was to keep the proper hardware relationships known while trying to limit identifier bias. There is a need to have adequate balance between abstraction and context retention so that the models perform stably.

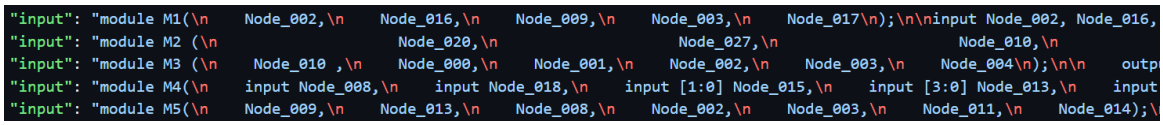


Figure 3.3: RTL Signal Obfuscation

3.4 QLoRA-Based Domain Adaptation

Full fine-tuning of large language models can require large computing resources due to the large number of trainable parameters. Typical methods of training these models require using multiple high-memory GPUs and taking a long time to train. As a result, there are significant practical limitations on hardware security research environments with restricted facilities.

To address these issues, the proposed framework uses Quantized Low-Rank Adaptation (QLoRA) to provide parameter-efficient fine-tuning [9]. QLoRA uses low-rank adaptation methods and 4-bit quantization to greatly reduce memory demand during training while preserving nearly all models' representational capability.

The proposed method uses NF4 4-bit quantization for loading a base transformer model via the bitsandbytes framework [9]. Instead of retraining the entire parameter space, low-rank adapter matrices are added to select transformer layers that can be optimized during training, while the original model weights remain frozen in quantized form.

This approach has numerous benefits. First, the reduced memory footprint allows training on consumer-grade GPUs, such as NVIDIA T4 hardware. Secondly, because only a

small number of parameters will be updated, training time will be reduced. Finally, the combination of QLoRA and domain-specific RTL training allows for the practical deployment of LLM-based hardware security systems in low-resource environments.

3.5 Training Strategy and Dataset Splitting

The prepared dataset is split into stratified sampling sets, which creates three sets: Training, Validation, and Test. Stratified sampling expects the ratio of clean and Trojan-infected samples to remain roughly constant across all partitions of the dataset.

The splits for the dataset are 70% of the dataset is used for Training, 10% is used for Validation, and 20% is used for Testing. The training dataset is for adapting the model; the validation dataset aids in monitoring learning stability and tuning experiment performance. The testing dataset is completely isolated from training to ensure there is no biased evaluation result.

The training methodology used is supervised fine-tuning with RTL prompts and their corresponding labels as inputs to the QLoRA-adapted model [9, 13]. The learning process is based on cross-entropy loss optimization, and during the training period, various evaluation metrics are tracked. The batch size and learning parameters are carefully chosen to provide an appropriate balance between the stability of the training process and the memory limitations of the hardware.

The framework uses mixed precision optimization and CUDA acceleration for the purpose of enhancing computational efficiency when conducting experiments [10]. Given that the dataset contains a wide range of possible RTL configuration structures, the model can gradually learn to build contextual hardware representations to identify suspicious Trojan related patterns.

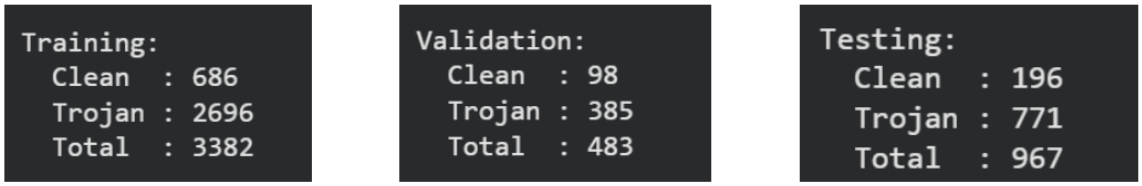


Figure 3.4: Stratified Dataset Splitting for Training, Validation, and Testing

3.6 Trojan Detection and Localization

The final phase of the proposed framework involves detection and localization of hardware Trojans in RTL designs. Detection involves determining whether or not a design has malicious Trojan behaviour, while localization is designed to assist in identifying suspicious trigger and payload regions within a design, which may be responsible for activating a Trojan, or performing a malicious function.

Once the model has been sufficiently trained, it will evaluate RTL inputs and provide an indication of whether the circuit is clean or infected by a Trojan virus. In addition to providing a classification output, the framework will also evaluate the structural regions of the RTL input that are associated with trigger logic or payload circuitry. The ability to localize these areas is essential for explainability and helps to assist hardware engineers during the process of verification and debugging.

Localization evaluation is defined through two primary metrics: Trigger Line Coverage (TLC) assesses the fraction of actual Trojan trigger lines accurately identified by the framework, whereas Payload Line Coverage (PLC) measures the percentage of malicious payload-related logic found in the hardware design that the model correctly identifies.

Localization metrics add a level of insight into model performance beyond what basic classification accuracy can provide. Even though classification results may be high, localization evaluations require further assessment to help determine if a model truly comprehends the characteristics of suspicious hardware or if it simply relies on the statistical correlations within the dataset used to train the model.

3.7 Design Choices and Justification

Multiple key design decisions were made during the creation of this framework. The choice of a transformer-based language model was based on its ability to model long-distance context from RTL code. Unlike prior traditional machine learning algorithms that must build feature sets by hand, transformer models learn about the sequential and relational structures of data more effectively and without the need for a great deal of hand-crafted feature engineering.

QLoRA was used for both hardware capability constraints and for model training effectiveness. Many of the models created in current research literature have used full fine-tuning on multiple RTX class graphics processing units with very high VRAM quantities. In contrast, QLoRA enables effective allocation of model parameters without sacrificing performance costs on a significantly smaller number of compute resources.

Adding structural obfuscation improves resilience against adversarial modifications and helps to reduce overfitting at the identifier level. It also encourages the model to learn real hardware relationships, rather than being biased towards hasty visual similarity among training examples

Adding both detection and localization metrics also improves interpretability and the depth of evaluation of detection and localization metrics. Hardware Trojan detection systems must classify malicious devices correctly and give useful information about why suspicious circuits may be hardware Trojans to make the recommended model for real hardware security analysis/verification practical.

Chapter 4

IMPLEMENTATION

4.1 Hardware and Software Environment

The experimental use of a software-based solution (QLoRA) to implement a hardware Trojan detection framework is based on a test bed created to test QLoRA in a low resource environment. Because one of the key objectives of this work is to provide a means to facilitate the development of QLoRA-enabled systems for use in consumer electronics, the design of the training and test workflows also met the limitations of available GPU memory resources while providing sufficient learning stability.

The majority of the experimentation was performed in Google Colaboratory using NVIDIA T4 GPUs with 16 GB VRAM. In contrast to conventional full fine-tuning methods, which generally require multiple high-performance Tesla RTX 3090 Ti or A100 class GPUs, the proposed circuit was successfully implemented using far fewer hardware resources than are typically used for full fine-tuning.

To enhance the computational throughput and reduce the processing time, model training and inference were accelerated by the use of CUDA.

The programming language Python was selected to implement all the components of the overall implementation because it has extensive library support in the areas of ML, NLP, and hardware description analysis. PyTorch was chosen as the primary deep learning framework for the research because it allows for flexible implementation of Transformer-based architectures and can utilize the flexibility of Transformer-based architectures. Open-source libraries were incorporated into the workflow for processing, model adaptation, quantization, and evaluation of models.

The complete implementation environment consists of the following libraries: PyTorch; CUDA Toolkit; PEFT; Unsloth; bitsandbytes; scikit-learn; PyVerilog. QLoRA-based parameter-efficient fine-tuning was implemented with PEFT. Bitsandbytes was used to perform NF4 4-bit quantization to reduce GPU memory usage. PyVerilog was used for RTL parsing and hardware description preprocessing. Scikit-learn was used to split datasets and compute evaluation metrics during experimentation.

4.2 RTL Dataset Preparation Module

The first major implementation module is dedicated to dataset preparation and preprocessing of RTL designs. Hardware descriptions collected from TrustHub and ASIC benchmark repositories were provided in raw Verilog format with multiple inconsistent structural styles, indentation styles, and naming conventions. These inconsistencies must be addressed before the language model can be trained.

The preprocessing pipeline consists of Python-based parsing scripts using standard PyVerilog utilities to parse each RTL sample into important components (modules, wires, registers, assignments, conditional statements, sequential blocks of logic). Comments and unnecessary format characters were removed from the RTL samples to create consistency within the dataset as much as possible.

The second major component of this implementation was converting RTL representations of hardware design into structured textual prompts that could be used for transformer-based training. The generated prompts included representations of the hardware code and indications of whether the design was clean or infected with a Trojan. This format for creating the prompts allowed the language model to treat the RTL structure like natural language sequences.

Preprocessing also included validation code for corrupted RTL files or incomplete hardware descriptions. Since benchmark datasets acquired from various sources could have different formatting structures, normalization procedures were added to maintain consistency throughout training.

The obfuscation module was added to strengthen the framework against hardware modifications that might be done maliciously. In practice, attackers often rename internal signals and change the structure of identifiers so that automated detection systems will not detect Trojan behavior. Thus, models trained directly on raw RTL data may learn names only as an overfit rather than as an actual representation of the circuit's behavior.

To overcome this limitation, signal names, registers, and internal wire identifiers were replaced with generic node-based identifiers as part of preprocessing [1, 3]. In order to replace hardware identifiers and keep the functionality of the circuit intact, Python scripts were created to automatically scan the RTL files for all hardware identifiers and replace them.

The obfuscation process was implemented very carefully, ensuring that any important structural dependency that is part of the hardware design stays intact. Excessive abstraction may diminish the context information needed during the learning process, while too little obfuscation will allow the model to memorize patterns in the text. Therefore, a balance was struck between the preservation of structural features of the hardware design and the use of abstraction at the identifier level.

Experimental results from previous training stages suggest that the use of obfuscation-aware preprocessing in RTL samples improved the model's ability to generalize to previously unseen RTL samples. The obfuscated model was significantly less dependent on

exact token sequence and was amongst the best performing models when tested against structurally altered Trojan implementations.

Pre - processed dataset


```
{"Instruction": "\nYou are tasked with designing a module for a simple calculator that can perform basic arithmetic operations. The
{"Instruction": "Design a module that can detect any edge in an 8-bit binary vector and output the binary value of the vector one c
{"Instruction": "Please act as a professional Verilog designer. Design a pipelined module that implements a 4-to-2 priority encoder
{"Instruction": "Create a new module that combines the functionality of the given modules to produce a system that stores 8-bit bin
{"Instruction": "Please act as a professional Verilog designer. Your task is to create a Verilog module that implements a clock gat
{"Instruction": "\n\nYou are tasked with designing a module that takes in a 4-bit binary number and outputs its decimal equivalent.
```


Obfuscation of Pre - processed dataset


```
"input": "module M1(\n    Node_002,\n    Node_016,\n    Node_009,\n    Node_003,\n    Node_017\n);\n\ninput Node_002, Node_016,
"input": "module M2 (\n                    Node_020,\n                    Node_027,\n                    Node_010,\n
"input": "module M3 (\n    Node_010 ,\n    Node_000,\n    Node_001,\n    Node_002,\n    Node_003,\n    Node_004\n);\n\n    outp
"input": "module M4(\n    input Node_008,\n    input Node_018,\n    input [1:0] Node_015,\n    input [3:0] Node_013,\n    input
"input": "module M5(\n    Node_009,\n    Node_013,\n    Node_008,\n    Node_002,\n    Node_003,\n    Node_011,\n    Node_014);\
```


Figure 4.1: RTL Preprocessing and Prompt Generation

4.3 QLoRA Fine-Tuning Pipeline

The primary component of the framework that has been developed is the adaptation of a pre-trained transformer network specifically for the purpose of detecting hardware Trojans. Rather than performing the typically computationally intensive full fine-tuning process, the current implementation relies upon a method of efficiently adapting parameters through the use of trainable low-rank matrices that can be inserted in layers of the transformer network.

The base transformer model was obtained from the bitsandbytes library and was loaded with NF4 4-bit quantization. Quantization substantially reduced the GPU memory requirements necessary to store and utilize the original transformer while still retaining the majority of the representative capabilities of the original transformer model. PEFT was then used to attach Low-Rank Adaptation (LoRA) modules to certain layers of attention throughout the transformer model.

The light weight adapter weights were trained only during training to allow for adaption of the quantized model weights, leaving them fixed. This significantly reduced the number of parameters that needed to be trained allowing stable fine-tuning with limited GPU resources. In addition the training optimization structure of Unsloth was used to assist with optimizing the overall training speed and memory efficiency during experimentation with the model.

During the training process, the pipeline for training included the phases of Tokenizing Inputs, Creating Batches of Inputs for training, Optimizing the Gradients of the Model, Saving Checkpoints of the training, and Monitoring Validation. Further Training (Cuda Acceleration) Optimization was performed during training by using Mixed Precision computation to help optimize throughput of the overall training process. Additionally, to ensure that the model was learning properly and not overfitting to the training data, loss and eval-

uation metrics were monitored periodically throughout the training process.

4.4 Training Configuration and Hyperparameters

The training Configuration was selected based on available hardware resources for use and on dataset characteristics. The RTL (Register-transfer-level) hardware datasets were considered to be very heterogeneous in terms of the structure of the hardware devices in the RTL dataset; due to the above-mentioned observations of HW devices, training stability and memory optimization were also ones needing to be considered during the HW device parameter selection.

The entire RTL dataset was composed of 4832 RTL HW devices. The RTL HW devices were then segmented into training, validation, and testing samples using stratified sampling. Approximately 70% of the dataset was used for training, 10% was used for validation and the remaining 20% was set aside to test the final Model developed. The use of stratified sampling ensured that a balanced viewpoint of the Number of clean and Trogans across all segments of the dataset is present.

Strategies such as gradient accumulation and a learning rate schedule were proposed and used to enable stable optimization in low GPU memory scenarios. Careful selection of batch sizes aimed at eliminating CUDA out-of-memory errors while ensuring reasonable training throughput.

Training was done entirely using NVIDIA T4 GPUs in Google Colab environments. The lowered memory consumption of the implemented QLoRA pipeline relative to full fine-tuning methods that require large amounts of VRAM illustrates its ability to maintain sufficient domain adaptation via effective means.

4.5 Trojan Detection and Localization Module

The final implementation stage of the project was the use of the fine-tuned domain-specific LLM to detect and localize hardware Trojans. During the inference phase, the LLM produced a binary classification of whether or not the hardware design was free from Trojan infestation based on the RTL prompt inputs.

In addition to providing a binary classification, the LLM also provided localization information for the potential trigger and payload areas within the RTL code. Trigger logic refers to the hardware-enabled conditions that will allow the malicious behavior to occur, whereas payload logic refers to the actual harmful functionality that occurs upon activation. By providing hardware engineers with the localizing ability, the LLM aids in increasing the interpretability of the RTL code, and assists with debugging and verification of the hardware.

Custom evaluation scripts were developed to calculate Trigger Line Coverage (TLC) and Payload Line Coverage (PLC). These metrics are used to compare the model's predicted

suspicious regions with the actual locations of Trojans found in benchmark RTL samples as a more in-depth look at how well the model understands Trojan behavior structurally.

Logging and visualization routines were also part of the implementation to save prediction outputs, training statistics, and localization analysis results. The results from the experiments proved that this architecture was able to identify suspicious hardware patterns while consuming much less computational power than traditional full fine-tuning methods.

4.6 Tools and Libraries Used

During the implementation of the architecture, various open-source tools and libraries were incorporated throughout the architecture. Each tool/library serves its own purpose in allowing for the hardware Trojan detection system to be preprocessed, trained, evaluated, and deployed efficiently.

PyTorch was the primary deep-learning framework selected due to its flexibility and support for transformer-based architectures [10]. The CUDA Toolkit provided GPU acceleration for training and inference. The PEFT Library was used for the implementation of QLoRA adaptation modules [13]; bitsandbytes enabled the use of 4-bit NF4 quantization [9].

PyVerilog has been used extensively for the RTL parsing and structural prepocessing throughout the entire processes involved in using the library to retrieve hardware constructs and to automate the parsing of verilog based circuit elements [12]. During the experimentation phase, the usage of Scikit-learn for splitting the datasets, computing and processing metrics to aid in evaluation contributed greatly to the completion of the search methodology [14].

Google Colab had been identified as the main execution environment for developing and conducting the experimental work, primarily because it was so easy to use, and which includes the use of GPU capabilities. Consequently, the combined integration of these programmes provided a fully developed low-resource hardware Trojan detection extension that can be used by academic researchers.

Chapter 5

RESULTS AND ANALYSIS

5.1 Experimental Setup

To measure how well the QLoRA framework works for detecting and locating hardware Trojans, we ran experiments to evaluate both the accuracy of detection and the efficiency of computation when operating in low-resource hardware environments. We focused on memory usage, training time, and the model's ability to withstand changes to the structure of the RTL design.

The experiments were conducted on Google Colaboratory using the NVIDIA T4 GPU, which has 16GB of VRAM [9]. We chose this environment to test whether advanced transformer based hardware security frameworks could work well with consumer grade computing types of equipment. CUDA and mixed precision optimizations were enabled during both training and inference to increase computation throughput.

The experimental data set consisted of 4832 RTL samples obtained from TrustHub and ASIC Benchmark repositories; 980 clean hardware designs; and 3852 hardware designs infected by Trojan. The characteristics of the dataset were chosen to represent a diverse sample of structures represented in the RTL samples, thereby improving the model's ability to generalize and produce reliable evaluations.

The dataset was divided into three parts (i.e., training, validation, testing) using stratified sampling techniques to maintain the same class distribution across each subset. Approximately 70% of the samples were used for training, 10% for validation and 20% for testing, resulting in a balanced evaluation.

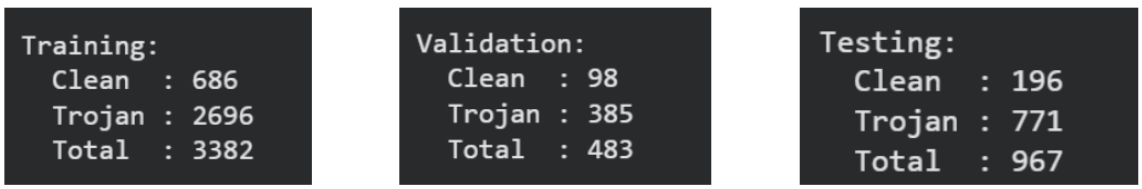


Figure 5.1: Stratified Dataset Splitting for Training, Validation, and Testing

5.2 Training Performance Analysis

The QLoRA framework performed stablely during the fine-tuning process despite having limited GPU memory. In the first few epochs of fine-tuning, the model started to build hardware-specific contextual relationships from the RTL prompt representations. In addition, the validation performance of the fine-tuning grew consistently over time and showed evidence of successful domain adaptation.

An important finding from the experimentation was the significant reduction of memory consumption achieved by using 4-bit quantization and low-rank adaptation. When performing traditional full fine-tuning on transformer models, high levels of virtual memory (VRAM) capacity are generally required since all transformer model parameters are able to be trained. In contrast, the proposed method updated only the lightweight adapter parameters of the model while keeping the frozen quantized weights.

When analyzing the training time of the proposed QLoRA pipeline method, it became evident that it provided substantial computational improvements over previous full fine-tuning approaches reported in literature. Even though the size of the fine-tuning dataset used was significantly larger than previous studies, the QLoRA-based pipeline was able to complete the fine-tuning process within an acceptable timeframe with one NVIDIA T4 GPU. Therefore, the results of the study demonstrated the scalability and accessibility of parameter efficient adaptation methods for conducting hardware security research.

Although minor fluctuations in validation loss were observed at certain intervals throughout the fine-tuning process and were attributed to the structural differences throughout the RTL dataset, the overall trend of learning remained consistent and there were no signs of significant (severe) overfitting displayed throughout the experimentation period.

5.3 Detection Performance Evaluation

The evaluation of the fine-tuned domain-specialized LLM utilized standard classification metrics (precision, recall, F1-score, and overall detection accuracy) [1, 3] to determine how well the framework could distinguish between clean and Trojan-affected circuits.

The experimental outcomes from the current proposal demonstrated that the newly proposed framework was capable of learning meaningful structural relationships within RTL descriptions and delivered reliable performance in detecting RTL designs across various hardware samples. The precision metric showed that the model provided many fewer false positive predictions than expected, while recall performance exhibited a very high level of sensitivity to the detection of malicious Trojan-infected designs.

The results of the F1-score indicated a balanced comparison of the precision and recall scores. Since the detection of hardware Trojans is characterized by very imbalanced datasets and the use of many different types of Trojans, the F1-score is an important measurement of the model's robustness. The experimental findings showed that even though

the FPGA-based QLoRA-adapted framework was validated using structurally obfuscated RTL, the framework's performance was stable.

The findings of this research study also highlight that the contextual learning provided by transformers has several advantages when compared to traditional feature-engineered machine learning techniques. Instead of relying solely on manually defined hardware features, the domain-specific LLM was able to learn the sequential and structural RTL dependencies directly from the training sample data.

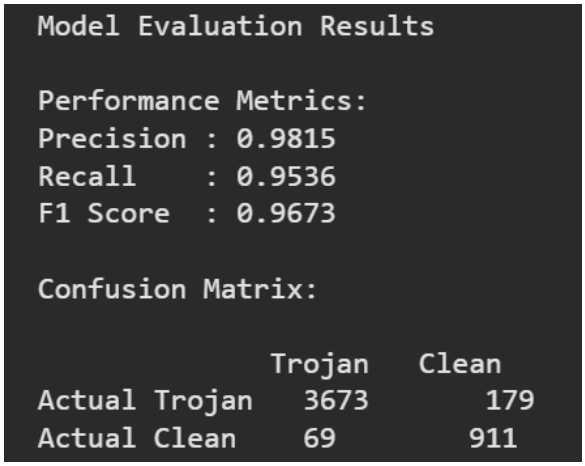


Figure 5.2: Detection Performance Metrics (Precision, Recall, F1-Score)

5.4 Trigger and Payload Localization Analysis

The framework evaluated at identifying binary Trojan detections also evaluated in terms such that they use TLC (Trigger Line Coverage) and PLC (Payload Line Coverage) in their capabilities of localizing and showing how well the behaviour of Trojans can be identified for how they are triggered to do their malicious intent through use of circuits, and so on (give examples).

TLC measures the percentage of actual lines responsible for triggering an RTL circuitry are detected (the equivalent in layman's terms): from finding real world examples of triggerable events that rarely occur and that are hidden in plain view by being dormant when circuits are functioning normally, it is critical to effectively identify said structures, as they often have been purposefully hidden by Trojan-creator in the form of complex hardware structures.

PLC measures the accuracy of correctly identifying payload circuit logic responsible for the circuit itself behaving properly (after trigger activated) because the logic has done "bad" things. The model had demonstrated an ability to highlight several regions of the circuit that contained logic that could be of concern in regards to Trojans, so the framework's ability to localize will assist the security and verification of circuits by inspecting the reason for a specific type of behaviour after an event occurs (trigger activated).

There are minor differences in localization performances from Trojan to Trojan architecture designs; however, the overall TLC and PLC results still support the conclusion that

the model learned enough related structures/tasks to indicate that there were meaningful hardware relationships that were developed and that were exhibited in the hardware after the hardware had completed the triggering process. This demonstrates that the framework has developed a partial structural understanding of how Trojans behave in an RTL description.

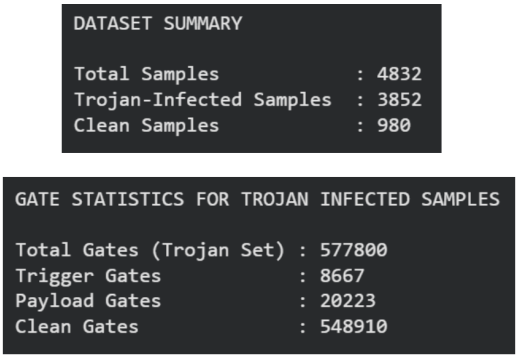


Figure 5.3: Gate count for TLC and PLC

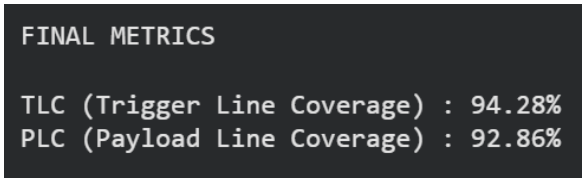


Figure 5.4: TLC , PLC performance metrics

5.5 Efficiency Comparison with Baseline Approaches

An evaluation was carried out comparing the performance of the alternative QLoRA framework to the standard methods of tuning, as described in other studies. This evaluation compares how much VRAM has been used during training; duration of training, effectiveness in using different datasets, and how much hardware was required to perform these training and/or tuning processes.

In general, the baseline tuning methods rely on only fine-tuning the entire model (i.e., full), and they also tend to use larger transformer models (e.g., LLaMA) along with multiple NVIDIA RTX 3090 Ti graphics cards that require a significant amount of VRAM to perform adequately. In general, while the baseline tuning methods produced very strong detection results, they required a great deal of costly computational infrastructure and took a long time to train the models. Therefore, baseline tuning methods reduce accessibility to the model training/tuning process for many academic and research environments, mainly due to financial considerations.

The proposed framework reduces VRAM consumption considerably through the use of NF4 quantization (4-bits) and low-rank adaptation for the framework structure [9]. Based on experimental results, the entire adaptation process was able to be performed successfully

with approximately 5.8 GB of VRAM on a single NVIDIA T4 GPU, whereas the baseline methods required substantially more VRAM.

The results from the evaluation of the time required to train the model were also significantly less with the proposed QLoRA framework, even with the dataset that had greater than 12 time as many sample points than some of the previously mentioned baseline models. The proposed framework, therefore, shows significant advantages as being able to provide robust performance per hardware requirements and robust performance, per hardware security application, and ease of scalability, even with using smaller and more efficient hardware.

Metric	**Baseline (Base Paper)**	**Proposed (QLoRA-Based)**
GPU Used	4 NVIDIA RTX 3090 Ti (96 GB VRAM)	NVIDIA T4 GPU (16 GB VRAM)
Model Setup	Full Fine-Tuning (LLaMA 3.1)	QLoRA-Based Adaptation
Dataset Size	4832 samples	4832 samples
Memory Usage	~11.41 GB VRAM	5.8 GB VRAM
Training Time	130464 Seconds (~36.2 hours)	13839.20 seconds (~3.8 hours)

Scaling Factor : 4832 / 400 = 12.08

Figure 5.5: GPU Memory Usage and Training Time Comparison

5.6 Discussion

There are many benefits associated with using parameter-efficient domain adaptation for detecting hardware Trojans, according to experimental outcomes [1, 3, 9]. The proposed combination of QLoRA and transformers has enabled effective learning of RTL structure while greatly lowering the amount of computational and memory costs involved.

In looking at this research, an important finding was the deployment of an LLM-based hardware security framework on consumer-grade GPUs. Most previous work on fine-tuning transformers to enhance security has assumed that expensive multi-GPU systems would be necessary to create the new framework; however, the proposed solution demonstrates that developing enhanced security for future hardware security research is achievable through efficient adaptation strategies.

The use of a structural obfuscation strategy has impacted the model's ability to create more robust designs. The structural relationships learned by the model were based on generalized hardware constructs, rather than relying mainly on explicit signal names or textual identifiers, improving its ability to perform well with structurally altered RTL samples and lessening the risk of overfitting to dataset-specific naming conventions.

Despite the success of this research, there were still several limitations noted during the experimentation with this framework. The very subtle trigger conditions for some Trojans

made localizing them extremely difficult; and there may still be optimization required for larger transformer architectures to effectively process highly complex SoC designs at an industrial scale. However, at a minimum, this work provides a solid basis for future research on low-resource hardware security through the use of domain-specific languages.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Summary of Contributions

The project describes a QLoRA-based domain-specific large language model framework that is enhanced for the ability to reliably detect and locate hardware Trojans in RTL design systems. This work was motivated by the increasing security issues presented by modern semiconductor supply chains and the widespread use of third-party hardware integration and technologies; thus, we achieved a resource-efficient transformer-based detection framework that is suitable for use in commercial-grade computational platforms and systems.

To accomplish our goals, we developed a full-scale pre-processing and structural encoding pipeline designed to process clean and Trojan-infected RTL samples obtained from both the TrustHub and ASIC benchmark repositories. The pre-processing phase worked to normalize all hardware description representations and convert RTL representations into structural prompts that can be consumed by transformer-based learning systems. Furthermore, structural obfuscation techniques were also applied during the pre-processing phase to improve the overall robustness of the structural encoding pipeline against adversarial modifications such as: signal renaming and identifier manipulation.

We also implemented QLoRA-based parameter-efficient fine-tuning using NF4 4-bit quantization and low-rank-adaptation techniques, which allowed us to achieve much lower GPU memory consumption and training costs than were possible with traditional full parameter fine-tuning methodologies. Additionally, the performance of the proposed framework has been empirically validated to demonstrate that it can be successfully trained on NVIDIA T4 GPUs while providing comparable levels of detection and localization abilities as those provided by existing literature.

TLC and PLC Metrics Were Used for Trojan Localization Analysis Within This Work. In The Proposed Framework, Identifying Suspicious Trigger and Payload Areas in RTL Descriptions Allowed For More Insight Into the Behavior of the Model During Hardware Trojan Analysis than Only Using Their Binary Classification Accuracy for Evaluation. As Such, It Will Improve the Interpretability of RTL Descriptions and Offer Greater Insight

into Model Behavior During Hardware Security Analysis.

Experimental Results Showed That The Use of QLoRA with LLM Adaptation Supported Efficient Hardware Trojan Detection Tasks, Resulting in Good Performance at a Much Lower Cost than Traditional Hardware Trojan Detection Methods, Making Future Research on Hardware Security More Accessible.

6.2 Limitations

The empirical results achieved through the proposed framework were satisfactory; however, a number of limitations were encountered in the course of performing experimentation. One of the core restrictions lay in the dependence upon existing benchmark datasets. While the dataset used in this study contained substantial numbers of digitally synthesised representative RTL implementations that were structurally different, the real-life performance metrics derived from real-world industrial applications and proprietary architectural designs are much more complex and potentially include many proprietary variations that may not be captured in publicly available benchmarking databases.

Another limitation of this study is the accuracy of localisations of Treachery (in this study) in terms of stealthy triggering conditions. Most forms of treachery contained in Trojan Designs are implemented with infrequently used triggering logic, i.e. usually only found in a handful of devices, and are deviously inserted/transformed into the underlying structure of a design making it extremely difficult for the model to accurately localise together with the corresponding trigger or payload.

The focus of this project was to develop a framework that was primarily designed for RTL-level hardware designs in Verilog. Therefore, the framework was not designed and implemented with the understanding that RTL should eventually lead to the need for corresponding analysis of hardware descriptions created using other hardware description language syntaxes (HDLs) or for the creation of gate-level netlist analyses. Primarily due to the increasing numbers of RTL, however, one approach could lead to significant efforts (especially in production settings), to extend the analysis of the framework into lower levels of abstraction (i.e. under the assumption that structural complexity will continue but that token representation will be exactly the same).

Additionally, although QLoRA was used to significantly decrease the computation requirements of the work, large transformer architectures continue to need to be optimised for the real-time deployment of designs with much larger than average SoC design sizes within industrial settings, meaning that memory efficiency and inference latency could influence future applications for both real and practical deployment scenarios.

6.3 Future Scope

The proposed framework can build upon almost every aspect of its current capabilities through improvement and extension of research directions in a future work. An example of a research direction to pursue is combining Graph Neural Networks (GNN) with transformer architecture for better structural comprehension of patterns that pertain to the connectivity in hardware.

Another notable research direction is looking further into how the current framework can accommodate more hardware abstraction levels, such as gate-level netlist representations, configuration files for FPGAs, and physical layouts of chips. Broadening the applicability of the framework beyond traditional RTL descriptions may allow researchers and developers to leverage it throughout all of the various stages of a semiconductor's lifecycle.

Developing explainable artificial intelligence mechanisms to use with the current framework's ability to perform hardware Trojan analysis represents another area for promising future research. The hardware Trojan localization evaluations are currently based on two output metrics (TLC and PLC); however, future iterations of this framework can be developed to offer more explicit paths for reasoning, provide value-added tracing of any suspicious dependencies, and deliver visualization capabilities to hardware verification engineers.

Using this framework against malicious adversarial threats is one area that can still be enhanced significantly. Adversaries may attempt to develop structurally modified hardware Trojan designs specifically to elude AI-based detection systems.

Lastly, future implementations of this framework will also investigate lightweight on-device deployment opportunities and real-time hardware monitoring uses through optimized transformer inference pipelines. Advancements to the above areas will allow for practical use of AI-driven hardware security systems within embedded platforms and semiconductor verification environments.

REFERENCES

[1] R. K. Kundu, K. Khalil, E. Garcia, E. Grassia, P. Calyam, and K. A. Hoque, "PEARL: An Adaptive and Explainable Hardware Trojan Detection Using Open-Source and Enterprise Large Language Models," *IEEE Access*, vol. 13, pp. 133755–133772, 2025.

[2] D. Saha, K. Yahyaei, S. K. Saha, M. Tehranipoor, and F. Farahmandi, "Empowering Hardware Security with LLM: The Development of a Vulnerable Hardware Database," in *Proceedings of the IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2024, pp. 233–243.

[3] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "TrojanWhisper: Evaluating Pre-Trained LLMs to Detect and Localize Hardware Trojans," *IEEE Access*, vol. 14, pp. 30965–30983, 2026.

[4] V. T. Hayashi and W. V. Ruggiero, "Hardware Trojan Detection in Open-Source Hardware Designs Using Machine Learning," *IEEE Access*, vol. 13, pp. 53200–53217, 2025.

[5] G.-W. Wan, S.-Z. Wong, D. Liu, and X. Wang, "Assessment of Pre-Trained Large Language Models for Hardware Trojan Detection in RTL Designs," in *Proceedings of the IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–6.

[6] E. Castillo, P. Schaumont, and M. Tehranipoor, "Hardware Trojan Detection Using Side-Channel Analysis," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 2, pp. 921–934, 2021.

[7] Y. Jin and Y. Makris, "Hardware Trojan Detection Using Path Delay Fingerprint," in *Proceedings of the IEEE International Workshop on Hardware-Oriented Security and Trust (HOST)*, 2008, pp. 51–57.

[8] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[9] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 38–45.

[11] TrustHub Benchmark Repository, "TrustHub Hardware Trojan Benchmarks," [Online]. Available: https://trust-hub.org

[12] PyVerilog Development Team, "PyVerilog Toolkit for Hardware Design Processing," [Online]. Available: https://github.com/PyHDI/Pyverilog

[13] Hugging Face, "PEFT: Parameter-Efficient Fine-Tuning of Large Language Models," [Online]. Available: https://github.com/huggingface/peft

[14] Scikit-learn Development Team, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.